CU-Connect — Production Migration Assessment

Topic: What the backend uses, what converts to MongoDB, and whether the app can go on Vercel. *Advisory only — no code changes.*

---

1. What your backend actually uses

Your real stack (differs from CLAUDE.md's Postgres+Redis+React description; this reflects the actual code):

Piece	Tech	File(s)
Web framework	Express 4	server.js, routes/*
Auth	JWT (jsonwebtoken) + bcryptjs	routes/auth.js, middleware/auth.js
Database	SQLite via Node built-in node:sqlite (DatabaseSync) — local file cu-connect.db	db/database.js, db/seed.js
Real-time	Socket.IO	lib/realtime.js
Code IDE execution	child_process.spawn running local python/gcc/java	lib/runner.js, routes/execute.js
Academics logic	Bunk calc + SGPA/CGPA (pure JS)	lib/academics.js
Frontend	Static HTML, served by Express	Frontend/Landing Page/*.html

2. What converts to MongoDB

Your 6 SQLite tables become 6 Mongoose collections:

- users → **User**
- subjects → **Subject**
- attendance → **Attendance** { studentId, subjectId, date, status }
- grades → **Grade**
- leaves + leave_audit → **Leave** (audit trail as embedded append-only array)
- announcements → **Announcement**

The catch: every route uses **synchronous** SQLite calls (db.prepare(...).get()/.run()). Mongoose is **async**, so all 10 backend files must be rewritten to async/await. Integer auto-increment IDs become Mongo _id ObjectIds — the JWT payload { id } and all foreign keys (studentId, subjectId, authorId) change type. Mechanical but touches everything. You also need **MongoDB Atlas** (free hosted DB); you can't ship a local file to production.

3. Uploading to Vercel — the honest problem

Vercel runs **serverless functions**: short-lived, stateless, read-only disk, no persistent process. Three features break on it:

- **SQLite** — the .db file is wiped on every cold start and not shared between instances. Data vanishes. (This is the real reason you need MongoDB — your instinct is right.)
- **Socket.IO** — needs a long-lived connection; serverless functions die after each request. **Live attendance sync won't work.**
- **Code IDE (runner.js)** — Vercel has no gcc/python/java and a read-only filesystem. **The IDE can't run.** It is also unsandboxed today — a security risk anywhere.

So "full project on Vercel" is **not viable as-is** without dropping real-time sync and the IDE.

4. Suggestions

Recommended split (all free tiers, keeps every feature working):

- **Frontend (static HTML)** → **Vercel**
- **Backend (Express + Socket.IO)** → **Render** or **Railway** (persistent Node server)
- **Database** → **MongoDB Atlas** (free M0)
- **Code IDE** → disable in production via a feature flag now; later wire to hosted **Judge0** for sandboxed execution. Don't ship the local spawn runner.

If you insist on Vercel-only: (a) migrate to Atlas, (b) replace Socket.IO with polling or a hosted service (Pusher/Ably), and (c) turn off the IDE. You lose the two real-time/IDE features.

Production must-dos regardless of host:

- Move JWT_SECRET to a real env var/secret (hardcoded dev default now).
- Add CORS config for the split frontend/backend origins.
- Lock down the unsandboxed code runner before it faces the internet.
- Add input validation and rate limiting on auth routes.

Fastest realistic path: MongoDB Atlas migration → backend on Render → frontend on Vercel.